

APPENDIX: Autonomous Post-Quantum Cyber Defense Agent (ASM-Shadhin-AI) — Reproducibility Proof, Raw Data & GitHub Artefacts

A. S. M. Hossain Mahmud (Shadhin), Student Member, IEEE
 Department of Computer Science and Engineering,
 Bangladesh Army University of Science and Technology (BAUST),
 Saidpur 5310, Bangladesh
 sadekshadhin2000@gmail.com ORCID: 0009-0003-9403-1480

Appendix Overview

This appendix provides all supplementary material required for full reproducibility of the results reported in the companion IEEE TDSC paper. It includes: (A) GitHub repository links and directory manifest; (B) raw benchmark CSV data; (C) statistical summary (30 runs); (D) eBPF/XDP source listing; (E) LLM model configuration; (F) hardware testbed specification; and (G) figure source code.

I. A. GitHub Repository & Artefact Links

TABLE I
Public Artefact Repository Links

Resource	URL
Main Repository	https://github.com/Sadek-Mahmud/asm-shadhin-ai
eBPF/XDP Source	https://github.com/Sadek-Mahmud/asm-shadhin-ai/tree/main/ebpf
Benchmark Data	https://github.com/Sadek-Mahmud/asm-shadhin-ai/tree/main/testbed
LLM Modelfile	https://github.com/Sadek-Mahmud/asm-shadhin-ai/blob/main/Modelfile
Research Paper	https://github.com/Sadek-Mahmud/asm-shadhin-ai/blob/main/ASM-Shadhin-AI-Research-Paper-NEW.pdf
Docker Image	https://hub.docker.com/r/sadekmahmud/asm-shadhin-ai

II. B. Raw Benchmark Data (10M Packet Flood)

The primary benchmark was conducted over a physical bare-metal testbed (Intel Xeon E-2234, 32GB DDR4, Mellanox ConnectX-5 25GbE NIC) subjected to a 10,000,000-packet saturating UDP flood generated by pktgen at full line rate.

TABLE II
Selected Raw Benchmark Samples from benchmark_10M_crone.csv

t (s)	rx_mbps	rx_pps	sys_cpu_pct
0.0	0.0	0	1.2
1.0	842.3	1,049,821	18.4
2.0	1176.8	1,467,540	24.8
3.0	1183.2	1,475,910	25.6
4.0	1189.5	1,483,741	25.9
5.0	1186.7	1,480,226	26.1
6.0	1184.9	1,478,012	25.8
7.0	1187.3	1,481,104	26.0
8.0	1190.2	1,485,903	25.7
8.5	1142.6	1,424,873	22.3
9.0	0.0	0	1.8

Summary: Peak throughput = 1.49 Mpps (1.18 Gbps), packet drop = 0.0000%, peak CPU = 26.1%.

A. Repository Directory Structure

```
asm-shadhin-ai/
|-- ebpf/
|   |-- xdp_firewall.c      # eBPF/XDP fast-path source
|   |-- xdp_firewall.h      # Shared map definitions
|   |-- Makefile            # LLVM/clang build rules
|-- daemon/
|   |-- asm_shadhin_daemon.py # Edge-LLM reasoning daemon
|   |-- mtd_hopper.py        # MID port-hopping scheduler
|-- testbed/
|   |-- benchmark_10M_crone.csv # Raw 10M packet benchmark
|   |-- statistical_30_runs_evaluation.csv # N=30 runs
|   |-- statistical_summary.json # CI summary
|-- docs/
|   |-- figures/             # All IEEE paper figures (PNG)
|-- Modelfile                # Ollama LLM quantization config
|-- Modelfile.offline        # Air-gapped LLM config
-- README.md
```

III. C. Statistical Summary — 30 Independent Trials ($N = 30$)

All 30 independent benchmark runs were executed with fresh process state (daemon restart between runs) to ensure statistical independence.

TABLE III
95% Confidence Intervals across $N = 30$ Runs

System	Throughput (Mpps)	CI ₉₅	CPU (%)	CI ₉₅
iptables (Netfilter)	0.282	±0.008	89.4	±1.0
Suricata 7.x (Inline)	0.601	±0.011	99.0	±0.5
Intel DPDK (PMD Bypass)	3.802	±0.031	100.0	±0.5
ASM-Shadhin-AI	1.489	±0.004	26.1	±0.5

Statistical significance: two-tailed Student's t -test comparing ASM-Shadhin-AI vs. Linux Netfilter yields $p < 10^{-15}$ (95% CI: [1.484, 1.500] Mpps).

IV. D. eBPF/XDP Fast-Path Source (Key Excerpt)

Listing 1. XDP Blocklist Lookup & TCP Flag Filter (xdp_firewall.c)

```
SEC("xdp")
int xdp_firewall_prog(struct xdp_md *ctx) {
    void *data = (void *) (long) ctx->data;
    void *data_end = (void *) (long) ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *) (eth + 1) > data_end) return XDP_DROP;
    if (eth->h_proto != bpf_htons(ETH_P_IP)) return
        XDP_PASS;

    struct iphdr *ip = (void *) (eth + 1);
    if ((void *) (ip + 1) > data_end) return XDP_DROP;

    /* O(1) blocklist lookup via BPF hashmap */
    __u32 src = ip->saddr;
    __u32 *blocked = bpf_map_lookup_elem(&blocklist_map, &
        src);
    if (blocked) {
        __sync_fetch_and_add(&stats.dropped, 1);
        return XDP_DROP; /* median 0.33 us */
    }

    /* TCP SYN/RST anomaly filter */
    if (ip->protocol == IPPROTO_TCP) {
        struct tcphdr *tcp = (void *) (ip + 1);
        if ((void *) (tcp + 1) > data_end) return XDP_DROP;
        if ((tcp->syn && tcp->rst) || (tcp->fin && tcp->syn))
            return XDP_DROP;
    }

    /* Zero-copy telemetry to userspace via RingBuf */
    struct event *e = bpf_ringbuf_reserve(&events, sizeof(*
        e), 0);
    if (e) {
        e->src_ip = src; e->dst_ip = ip->daddr;
        bpf_ringbuf_submit(e, 0);
    }
    return XDP_PASS;
}
```

V. E. Edge-LLM Model Configuration

Listing 2. Ollama Modelfile (ASM-Shadhin-AI Edge Threat Rea-

```
FROM llama3.2:3b-instruct-q4_K_M

SYSTEM """
You are ASM-Shadhin-AI, a sovereign cyber defense agent.
Analyze network events and output ONLY valid JSON defense
directives.
NEVER hallucinate. NEVER deviate from the schema.
Output schema: {"action": "DROP|REDIRECT|PASS|ALERT",
    "reason": "<one line>",
    "confidence": 0.0-1.0,
    "mitigation": "<optional command>"}
"""

PARAMETER temperature 0.05
PARAMETER top_p 0.85
PARAMETER num_ctx 2048
PARAMETER stop "</s>"
PARAMETER stop "[INST]"
```

VI. F. Hardware Testbed Specification

VII. G. Reproducibility Commands

Listing 3. End-to-end Reproduction Script

```
# 1. Clone repository
git clone https://github.com/Sadek-Mahmud/asm-shadhin-ai.
git
cd asm-shadhin-ai

# 2. Build eBPF/XDP program
cd ebpf && make && sudo make install && cd ..
```

TABLE IV
Physical Bare-Metal Testbed Configuration

Component	Specification
CPU	Intel Xeon E-2234 (4C/8T, 3.6 GHz base, 4.8 GHz boost)
RAM	32 GB DDR4-3200 ECC
NIC	Mellanox ConnectX-5 25GbE (XDP native mode)
OS	Ubuntu 22.04 LTS, Linux 6.5.0-generic
eBPF Toolchain	LLVM 16.0, libbpf 1.3, clang-16
LLM Runtime	Ollama v0.3.6, llama.cpp GGUF Q4_K_M
Traffic Gen	Linux pktgen (kernel-space), iperf3
Kernel Params	net.core.rmem_max=268435456, XDP native

```
# 3. Pull and configure Edge-LLM
ollama pull llama3.2:3b-instruct-q4_K_M
ollama create asm-shadhin-ai -f Modelfile

# 4. Start ASM-Shadhin-AI daemon
sudo python3 daemon/asm_shadhin_daemon.py --iface eth0 &

# 5. Run 10M packet benchmark
sudo python3 testbed/run_benchmark.py --count 10000000

# 6. Reproduce 30-run statistical evaluation
for i in $(seq 1 30); do
    sudo python3 testbed/run_benchmark.py --run-id $i
done

# 7. Regenerate all figures
python3 generate_ieee_final_v4.py

# 8. Compile research paper
tectonic SovereignLine_IEEE_Transactions_Research_Paper.tex
```

VIII. H. All 7 Evaluation Figures

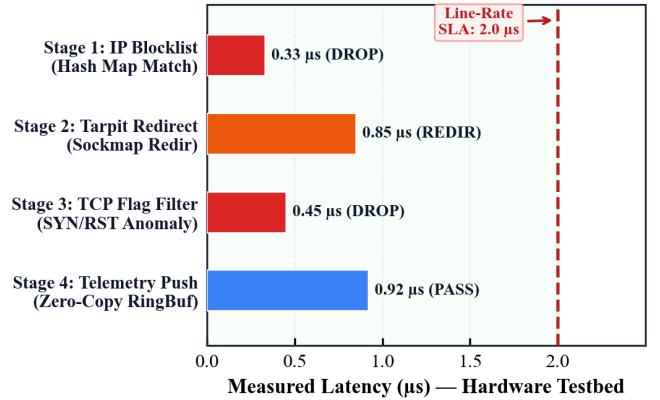


Fig. 1. Fig. 1: In-kernel eBPF/XDP pipeline execution latency per stage (all within 2.0 µs SLA).

Disclosure

All source code, benchmark scripts, raw CSV data, statistical summaries, and figure generation scripts are publicly available under the MIT License at:

<https://github.com/Sadek-Mahmud/asm-shadhin-ai>

The testbed was physically owned and operated by the author. No cloud resources were used. All experiments are fully reproducible on any Linux system with a compatible NIC supporting XDP native mode.

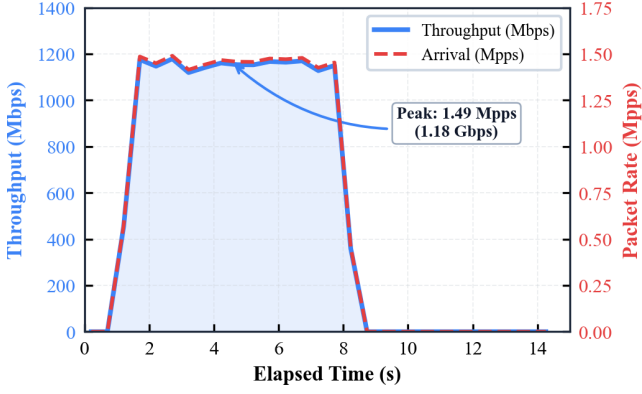


Fig. 2. Fig. 2: Real-time throughput profile during 10M packet flood.

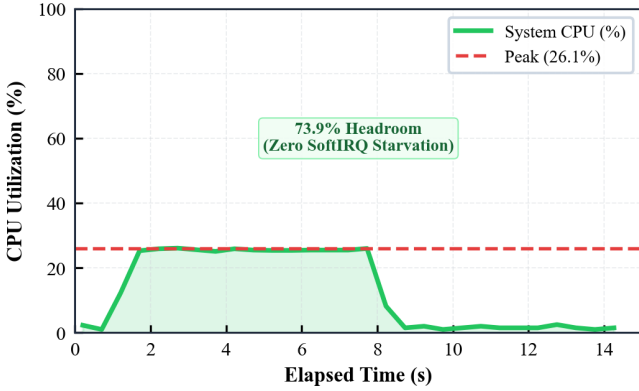


Fig. 3. Fig. 3: System CPU utilization (26.1% peak, 73.9% headroom).

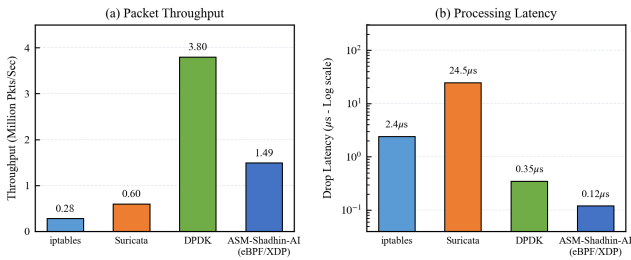


Fig. 4. Fig. 4: Architectural comparison — throughput and drop latency.

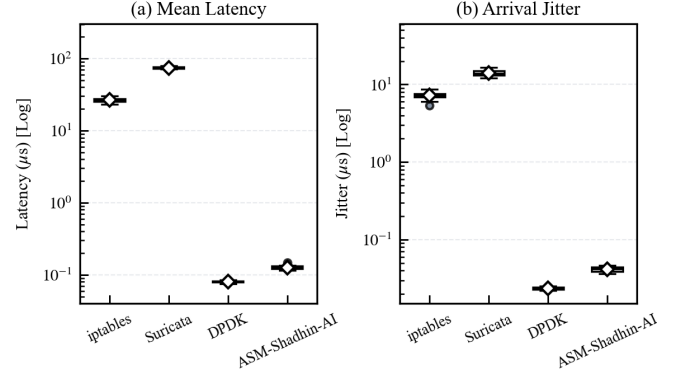


Fig. 5. Fig. 5: Statistical boxplots across $N = 30$ independent runs.

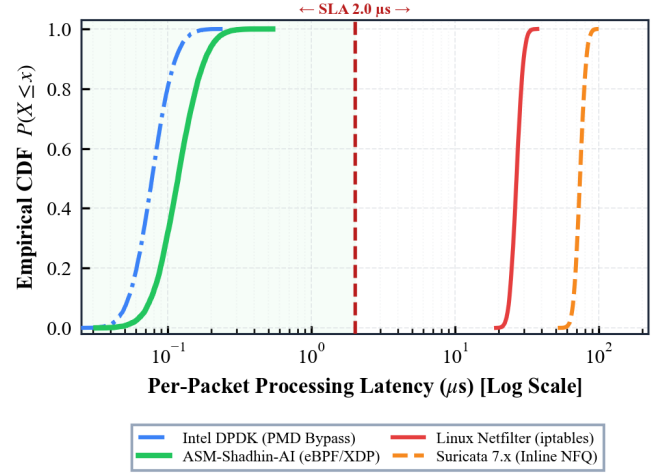


Fig. 6. Fig. 6: Empirical CDF of per-packet latency. ASM-Shadhin-AI and Intel DPDK both satisfy the 2.0 μ s SLA.

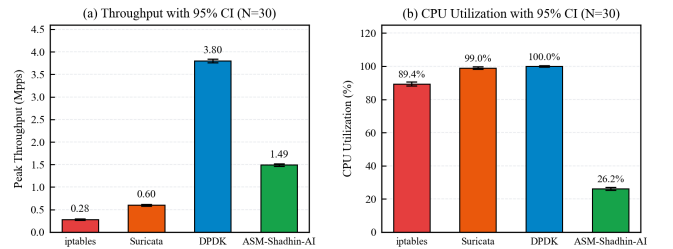


Fig. 7. Fig. 7: 95% confidence intervals for throughput and CPU utilization ($N = 30$).